

UNIVERSITÀ DEGLI STUDI DI SALERNO



DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED
ELETTRICA E MATEMATICA APPLICATA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Corso I-Z : Gruppo 31

Project Work: Embedded Systems

Docenti

Prof. Luca Greco

Prof. Gennaro Percannella

Prof. Vincenzo Carletti

Autore

Alessandro Maruotto

Sabato Paolillo

Anno Accademico 2024/2025

Indice

1	Introduzione al problema	3
1.1	Specifiche principali	3
1.2	Componenti utilizzati	4
1.3	Il V-Model: Un Approccio Metodico allo Sviluppo Embedded . .	5
1.3.1	Modello a V	5
1.3.2	Integrazione con Metodologie Model-Based	5
1.3.3	Stratificazione delle Attività di Testing	6
1.3.4	Vantaggi nell'Embedded Development	6
2	Requisiti del sistema domotico	8
2.1	Panoramica delle funzionalità	8
2.1.1	Circuito luci	8
2.1.2	Sistema di Riscaldamento	9
2.1.3	Sistema di gestione automatizzata delle veneziane	10
2.1.4	Comportamenti di sistema	11
2.2	Classificazione dei requisiti	12
2.2.1	Requisiti funzionali	12
2.2.2	Requisiti non funzionali	13
2.2.3	System Use Case	14
3	Progettazione della logica di sistema	18
3.1	Progettazione stateflow	18
3.1.1	Input, Output e variabili del sistema	18
3.1.2	Stato INATTIVO	20
3.1.3	Stato ATTIVO	20
3.1.4	Stato CONFIGURAZIONE	20
3.1.5	Stato OPERAZIONE	21
3.2	Implementazione simulink	21

4	Implementazione del sistema domotico	25
4.1	Schema di collegamento	25
4.2	Implementazione Simulink con STM32	27
4.2.1	Impostazioni ST CubeMX	31
4.3	Fase di testing	34
5	Conclusioni	36
5.1	Risultati Raggiunti	36
5.2	Funzionalità Implementate	36
5.3	Aspetti Tecnici	37
5.4	Innovazioni Introdotte	37
5.5	Validazione e Testing	37
5.6	Vantaggi dell'Approccio Model-Based	38
5.7	Considerazioni Finali	38

Capitolo 1

Introduzione al problema

Il progetto ha come scopo la progettazione di un firmware per un sistema domotico destinato alla gestione automatizzata di funzionalità in ambiente residenziale. In particolare, il sistema deve gestire:

- L'*apertura delle veneziane* tramite un servo motore;
- L'*accensione* e lo *spegnimento delle luci* mediante relè, con supporto di *temporizzazione* per lo spegnimento automatico delle luci;
- La *regolazione della temperatura* con feedback attraverso modulazione di led RGB.

L'interazione con il sistema avviene sia tramite pulsantiera fisica, sia tramite interfaccia *UART* che consente la configurazione dei parametri di sistema. Il dispositivo gestisce un insieme di stati operativi e fornisce feedback all'utente con led di segnalazione per indicare lo stato di attività del dispositivo.

1.1 Specifiche principali

- **Pulsanti disponibili**
 - **CONFIG**: abilita/disabilita la modalità di configurazione;
 - **ATTIVA**: attiva/disattiva il sistema;
 - **LUCI**: accende/spegne le luci;
 - **LUCI_TMP**: attiva l'accensione temporizzata delle luci;
- **Modalità di configurazione**
 - Utilizzabile solo se il sistema è attivo;

- Permette l'impostazione di:
 - * *Angolo delle veneziane*: da 0° (chiuse) a 180° (aperte), in passi da 20°;
 - * *Luci*: accese/spente; temporizzazione impostabile tra 5 e 30 secondi;
 - * *Temperatura*: da 16°C a 30°C, con risoluzione di 1°C;
- **Feedback e stato**
 - Il LED utente lampeggia a 2Hz se il sistema è attivo;
 - Il LED RGB segnala la temperatura selezionata con un codice colore;
 - Ogni 10 secondi, lo stato viene trasmesso via UART se il sistema è attivo e non in configurazione.
- **Avvio del dispositivo**
 - Il dispositivo parte in modalità inattiva e, in assenza di una configurazione valida, resta in stato di configurazione obbligatoria.

1.2 Componenti utilizzati

A seconda della funzionalità da implementare i componenti si differenziano:

- **Sistema:**
 - 2× pulsante;
 - 1× LED giallo;
- **Veneziane:**
 - 1× servo motore SG90;
- **Luci:**
 - 2× pulsanti;
 - 1× relè 5V;
- **Temperatura**
 - 1× LED RGB.

1.3 Il V-Model: Un Approccio Metodico allo Sviluppo Embedded

1.3.1 Modello a V

Quando si parla di sviluppo di sistemi embedded, la complessità deriva principalmente dall'intreccio indissolubile tra componenti hardware e software. Il V-Model emerge come risposta a questa sfida, proponendo un paradigma che va oltre la semplice sequenza lineare del modello waterfall tradizionale.

La caratteristica distintiva di questo approccio risiede nella sua struttura simmetrica: ogni fase di progettazione trova il suo corrispettivo in una fase di testing specifica. Questa corrispondenza biunivoca non è casuale, ma riflette una filosofia progettuale che considera la verifica non come un'attività conclusiva, ma come un processo parallelo e complementare allo sviluppo.

1.3.2 Integrazione con Metodologie Model-Based

L'adozione del V-Model si armonizza perfettamente con gli approcci *model-based design*, dove strumenti come Simulink e Stateflow permettono di costruire rappresentazioni matematiche del sistema target. Questa sinergia consente di:

- **Anticipare problematiche critiche** attraverso simulazioni virtuali prima della prototipazione fisica
- **Ridurre significativamente i costi** di correzione degli errori identificando anomalie nelle fasi iniziali
- **Garantire tracciabilità** tra requisiti di alto livello e implementazione dettagliata

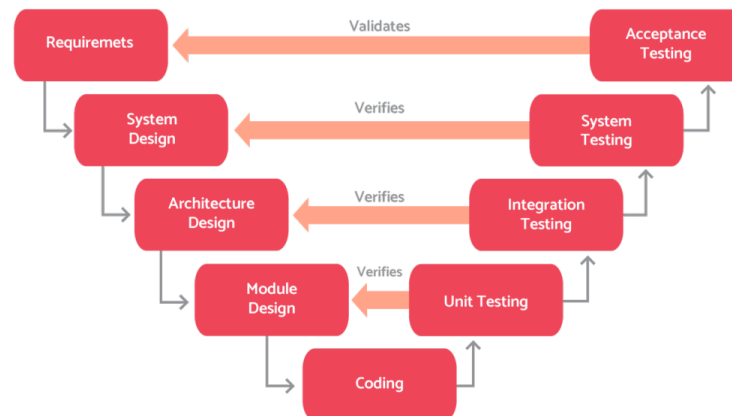


Figura 1.1: Modello di sviluppo software a V

1.3.3 Stratificazione delle Attività di Testing

La metodologia prevede quattro livelli distinti di verifica, ciascuno con obiettivi e scope specifici:

- **Unit Testing:** rappresenta il livello più granulare, concentrandosi sulla correttezza funzionale dei singoli componenti software. Qui l'attenzione è rivolta alla logica interna di funzioni, metodi e classi.
- **Integration Testing:** sposta il focus sull'interoperabilità, verificando che l'interfacciamento tra moduli software e la comunicazione con l'hardware avvengano secondo le specifiche progettuali.
- **System Testing:** adotta una prospettiva olistica, valutando se il comportamento emergente del sistema completo sia conforme ai requisiti funzionali e non funzionali stabiliti.
- **Acceptance Testing:** conclude il processo con la validazione finale, assicurando che il prodotto risponda efficacemente alle esigenze reali dell'utente finale e del contesto operativo.

1.3.4 Vantaggi nell'Embedded Development

Nel dominio dei sistemi embedded, dove margini di errore sono spesso minimi e i costi di rilavorazione elevati, il V-Model offre vantaggi tangibili attraverso la sua natura strutturata e la ridondanza dei controlli di qualità integrati nel processo stesso.

L'approccio metodico del V-Model permette di mantenere un elevato standard qualitativo durante tutto il ciclo di sviluppo, riducendo significativamente il rischio di difetti critici nelle fasi finali del progetto.

Capitolo 2

Requisiti del sistema domotico

2.1 Panoramica delle funzionalità

2.1.1 Circuito luci

Il circuito luci si compone di una combinazione una coppia di pulsanti (`LUCI` e `LUCI_TMP`) e un relè. La pressione del pulsante `LUCI` ha effetto immediato sullo stato delle luci, secondo queste regole:

- Se le luci sono SPENTE, vengono ACCESE;
- Se le luci sono ACCESE, vengono SPENTE;
- Se le luci sono ACCESE con ACCENSIONE TEMPORIZZATA, vengono SPENTE IMMEDIATAMENTE, interrompendo la temporizzazione.

La pressione del pulsante `LUCI_TMP` avvia l'accensione temporizzata delle luci. Il comportamento varia in base allo stato corrente:

- Se le luci sono SPENTE, vengono accese per la durata temporizzata impostata, poi si spengono automaticamente;
- Se le luci sono ACCESE in modalità MANUALE (non temporizzata), passano in modalità temporizzata: restano accese per la durata configurata e poi si spengono;
- Se le luci sono già ACCESE in modalità TEMPORIZZATA, non succede nulla (pressione del pulsante ignorata).

Ogni transizione tra accensione e spegnimento attiva la bobina del relè a 5V.

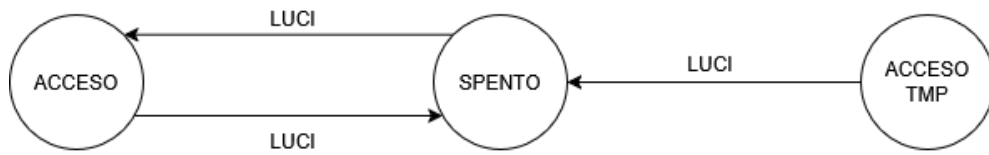


Figura 2.1: Conversione del pulsante LUCI in FSM.

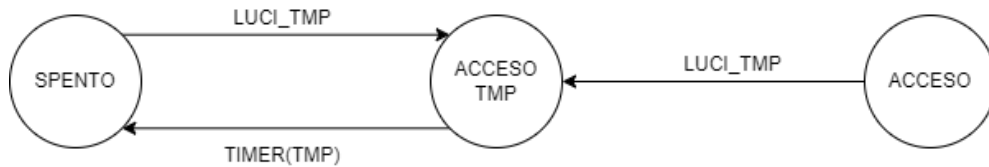


Figura 2.2: Conversione del pulsante LUCI_TMP in FSM.

La configurazione dell'accensione temporizzata sarà impostata in un range $[5 \div 30]$ sec, questo da utilizzare solo quando ci si trova nello stato di accensione luci temporizzate. Se il timer è impostato a 0, allora l'accensione temporizzata non viene effettuata.

2.1.2 Sistema di Riscaldamento

Il modulo di controllo termico realizza una simulazione del riscaldamento mediante regolazione fittizia della temperatura nell'intervallo $[16 \div 30]^{\circ}\text{C}$. Quando il dispositivo risulta inattivo, il sistema imposta automaticamente la temperatura ambiente simulata al valore minimo di 16°C .

La visualizzazione dello stato termico corrente viene effettuata attraverso un LED RGB che implementa una codifica cromatica specifica. L'intervallo termico complessivo è suddiviso in cinque sotto-intervalli distinti, ciascuno associato a una tonalità caratteristica come riportato nella Tabella 2.1.

Blu	Viola	Giallo	Arancione	Rosso
$[16 \div 18]^{\circ}\text{C}$	$[19 \div 21]^{\circ}\text{C}$	$[22 \div 24]^{\circ}\text{C}$	$[25 \div 27]^{\circ}\text{C}$	$[28 \div 30]^{\circ}\text{C}$

Tabella 2.1: Corrispondenza temperatura-colore per indicazione LED RGB

La generazione dei colori previsti presenta diversi livelli di complessità implementativa. Mentre alcune tonalità (blu, giallo, rosso) possono essere prodotte mediante l'attivazione esclusiva di un singolo canale del LED RGB, altre colorazioni richiedono necessariamente la combinazione di più canali con intensità differenziate. Tale approccio rende indispensabile l'utilizzo della modulazione PWM (Pulse Width Modulation) per il controllo preciso dell'intensità luminosa

di ciascun componente cromatico.

La codifica RGB standard definisce i seguenti valori numerici per le colorazioni implementate:

- Blu – codifica `RGB(0, 0, 255)`;
- Viola – codifica `RGB(127, 0, 255)`;
- Giallo – codifica `RGB(255, 255, 0)`;
- Arancione – codifica `RGB(255, 127, 0)`;
- Rosso – codifica `RGB(255, 0, 0)`.

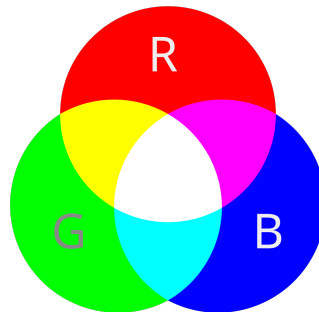


Figura 2.3: Schema delle combinazioni cromatiche nel modello RGB

L'analisi dei valori numerici evidenzia come la sintesi di tonalità complesse richieda la modulazione del *duty cycle* specifico per ciascun canale. A titolo esemplificativo, la generazione del colore arancione necessita del mantenimento del canale rosso alla massima intensità (100%), mentre il canale verde viene modulato al 50% della potenza nominale, corrispondente al rapporto tra il valore numerico 127 e il massimo teorico 255.

Data l'elevata complessità delle transizioni di stato e il numero considerevole di combinazioni possibili tra i parametri di controllo, la logica di gestione viene implementata mediante una funzione dedicata piuttosto che attraverso una tradizionale macchina a stati finiti realizzata in ambiente Stateflow.

2.1.3 Sistema di gestione automatizzata delle veneziane

La funzionalità di controllo delle schermature solari rappresenta l'ultimo modulo del sistema domotico implementato, realizzando la movimentazione automatica di una veneziana attraverso l'impiego di un servomotore dedicato. Il posizionamento della veneziana viene ottenuto mediante la variazione controllata del *duty cycle* applicato al segnale di comando del motore.

La correlazione tra l'ampiezza temporale dell'impulso di controllo e l'angolo di posizionamento risultante è illustrata nella Figura 2.4. Il servomeccanismo risponde a specifici valori di larghezza dell'impulso secondo la seguente caratterizzazione:

- Impulso da 1ms - posizionamento a 0° (veneziana completamente chiusa);
- Impulso da 1.5ms - posizionamento a 90° (apertura intermedia);
- Impulso da 2ms - posizionamento a 180° (veneziana completamente aperta).

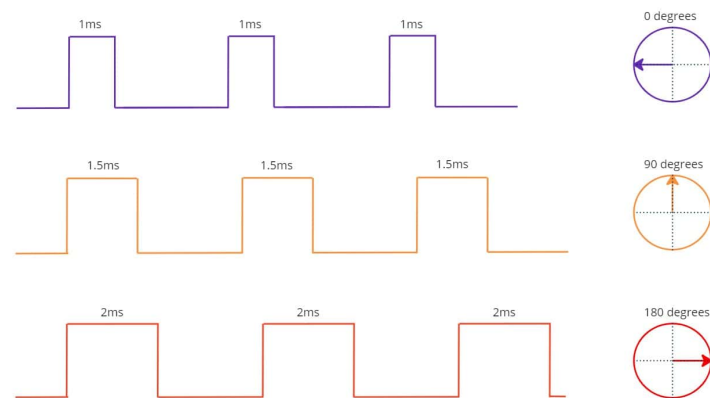


Figura 2.4: Relazione tra duty cycle del segnale di controllo e angolo di rotazione del servomotore SG90.

Le specifiche di progetto richiedono che l'angolazione della veneziana sia selezionabile esclusivamente attraverso un insieme discreto di posizioni predefinite, distribuite uniformemente nell'arco operativo $0^\circ - 180^\circ$ con incrementi fissi di 20° . La sequenza di posizionamenti ammissibili comprende quindi i valori: 0° , 20° , 40° , 60° , 80° , 100° , 120° , 140° , 160° , 180° , escludendo qualsiasi posizione intermedia non conforme a tale discretizzazione.

2.1.4 Comportamenti di sistema

Il comportamento del dispositivo cambia a seconda se si trova nello stato di primo utilizzo o meno. Al primo avvio del sistema esso si trova nello stato INATTIVO.

Non appena si preme il pulsante ATTIVA esso transita nello stato ATTIVO, e di conseguenza nello stato di CONFIGURAZIONE, dove è richiesta l'interazione con l'utente per l'impostazione dei parametri riguardanti le tre macro-funzioni elencate in precedenza. La selezione dei parametri deve avvenire tramite interfaccia UART. Poiché si è nella prima attivazione, l'utente non può scegliere quali

funzionalità settare, ma queste vengono presentate ad esso sequenzialmente. Fintantoché l'utente non inserisce una configurazione valida per tutte le funzionalità, esso non può uscire dallo stato di CONFIGURAZIONE (riferito alle sottosezioni 2.1.1, 2.1.2 e 2.1.3).

Se la configurazione iniziale è stata completata, le successive configurazioni non devono essere necessariamente completate. L'utente, infatti, può annullare l'inserimento del nuovo valore premendo il pulsante CONFIG e uscendo quindi dalla configurazione. Questo porterà alla persistenza degli attributi precedentemente impostati.

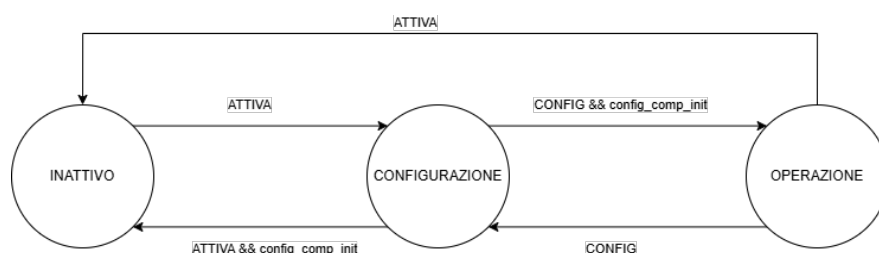


Figura 2.5: FSM riportante i cambiamenti di stato tra INATTIVO, CONFIGURAZIONE e OPERAZIONE.

Una volta conclusa la fase di configurazione, l'utente può passare allo stato OPERAZIONE, dove sono eseguite in parallelo le macro-funzioni. Inoltre, saranno richieste:

- La pulsazione di un LED utente lampeggiante ad una frequenza di 2Hz;
- La trasmissione su interfaccia UART, ogni 10sec, dei parametri attualmente impostati del sistema.

A dispositivo INATTIVO, riportare i parametri del sistema ai seguenti valori:

- Venezieane chiuse (0° servo motore);
- Luce spenta (uscita relè bassa)/Temporizzazione inattiva (pari a 0);
- Temperatura al minimo (16°, luce blu).

2.2 Classificazione dei requisiti

2.2.1 Requisiti funzionali

I requisiti funzionali specificano le operazioni che il sistema deve essere in grado di eseguire:

1. Controllo delle luci:

- Accensione e spegnimento manuale tramite pulsante `LUCI`.
- Accensione temporizzata tramite pulsante `LUCI_TMP`.
- Interruzione della temporizzazione tramite nuova pressione del pulsante `LUCI`.

2. Gestione della temperatura:

- Selezione della temperatura nel range 16–30°C tramite interfaccia UART.
- Visualizzazione della temperatura impostata tramite colore del LED RGB.

3. Controllo delle veneziane:

- Regolazione dell'angolo da 0° a 180°, con step di 20°, tramite UART.
- Conversione dell'angolo configurato in duty cycle PWM per pilotare il servo motore.

4. Gestione degli stati operativi:

- Stato iniziale: `INATTIVO`.
- Stato `CONFIGURAZIONE` per la configurazione dei parametri.
- Stato `OPERAZIONE` per l'esecuzione delle funzionalità.

5. Feedback visivi e comunicazione:

- LED utente lampeggiante a 2 Hz in stato `ATTIVO`.
- Invio dei parametri attivi via UART ogni 10 secondi.

6. Persistenza e reset dei parametri:

- Memorizzazione dei parametri dopo la prima configurazione.
- Reimpostazione dei valori predefiniti al ritorno nello stato `INATTIVO`.

2.2.2 Requisiti non funzionali

I requisiti non funzionali definiscono vincoli, caratteristiche qualitative e aspetti tecnici del sistema:

1. **Reattività:** il sistema deve rispondere prontamente alla pressione dei pulsanti;

2. **Configurabilità:** tutti i parametri devono essere configurabili esclusivamente tramite interfaccia UART;
3. **Robustezza:**
 - I valori non possono essere fuori range, devono essere inseriti correttamente;
 - Se il valore inserito non rispetta i requisiti, il sistema permette di reinserire il valore. La richiesta viene ripetutamente inviata fintantoché l'utente non inserisce un valore valido.
4. **Interfaccia utente:** interazione con l'utente tramite menu testuale via terminale (es. PuTTY);
5. **Modularità e scalabilità:** le tre macro-funzionalità sono gestite in modo indipendente e parallelo tramite Stateflow.

2.2.3 System Use Case

In questa sezione sono rappresentati alcuni casi d'uso del sistema comprendendo una breve descrizione, il flusso degli eventi e le relative pre/post condizioni.

Elemento	Descrizione
<i>Nome</i>	Accensione/Spegnimento manuale delle luci
<i>ID</i>	UC1
<i>Descrizione</i>	L'utente preme il pulsante LUCI per accendere o spegnere le luci.
<i>Flusso eventi</i>	1. L'utente preme il pulsante LUCI; 2. Se le luci sono spente, vengono accese; 3. Se le luci sono accese, vengono spente; 4. Se le luci sono accese in modalità temporizzata, la temporizzazione viene interrotta e le luci spente.
<i>Pre-condizioni</i>	Il sistema si trova nello stato OPERAZIONE.
<i>Post-Condizioni</i>	Stato delle luci aggiornato correttamente.
<i>Eccezioni</i>	-

Elemento	Descrizione
<i>Nome</i>	Accensione temporizzata delle luci
<i>ID</i>	UC2
<i>Descrizione</i>	L'utente preme il pulsante LUCI_TMP per accendere le luci temporaneamente.
<i>Flusso eventi</i>	1. L'utente preme il pulsante LUCI_TMP; 2. Se le luci sono spente, si accendono per un intervallo temporale configurato; 3. Se sono accese manualmente, passano in modalità temporizzata; 4. Se sono già accese temporizzate, la pressione è ignorata.
<i>Pre-condizioni</i>	Sistema in stato OPERAZIONE; timer temporizzato > 0.
<i>Post-Condizioni</i>	Le luci si spengono automaticamente al termine del tempo impostato.
<i>Eccezioni</i>	-

Elemento	Descrizione
<i>Nome</i>	Configurazione iniziale del sistema via UART
<i>ID</i>	UC3
<i>Descrizione</i>	L'utente imposta i parametri iniziali tramite interfaccia UART alla prima attivazione.
<i>Flusso eventi</i>	1. Il sistema si trova nello stato INATTIVO; 2. L'utente preme il pulsante ATTIVA; 3. Il sistema entra in stato CONFIGURAZIONE; 4. Viene richiesto l'inserimento dei parametri tramite UART; 5. Se i parametri sono validi la prima configurazione è stata completata, il sistema passa in stato OPERAZIONE tramite tasto CONFIG.
<i>Pre-condizioni</i>	Primo avvio del sistema.
<i>Post-Condizioni</i>	Parametri salvati; sistema attivo.
<i>Eccezioni</i>	Parametri non validi → il sistema richiede l'inserimento di un valore valido.

Elemento	Descrizione
<i>Nome</i>	Controllo apertura veneziane
<i>ID</i>	UC4
<i>Descrizione</i>	L'utente seleziona un angolo di apertura per le veneziane tramite UART.
<i>Flusso eventi</i>	1. L'utente imposta il valore desiderato tra 0 e 9; 2. Il duty cycle del PWM viene aggiornato; 3. Il servo motore si posiziona sull'angolo indicato.
<i>Pre-condizioni</i>	Sistema in stato CONFIGURAZIONE.
<i>Post-Condizioni</i>	Veneziane posizionate all'angolo richiesto quando si torna nello stato OPERAZIONE.
<i>Eccezioni</i>	Valore non valido → Il sistema richiede l'inserimento di un valore valido.

Elemento	Descrizione
<i>Nome</i>	Indicazione temperatura tramite LED RGB
<i>ID</i>	UC5
<i>Descrizione</i>	Il sistema rappresenta la temperatura selezionata mediante LED RGB.
<i>Flusso eventi</i>	1. L'utente imposta la temperatura tra 16°C e 30°C; 2. Il sistema determina il colore corrispondente; 3. Il LED RGB viene attivato con PWM per visualizzare il colore.
<i>Pre-condizioni</i>	Sistema in stato CONFIGURAZIONE.
<i>Post-Condizioni</i>	Colore LED aggiornato in base alla temperatura al ritorno in OPERAZIONE.
<i>Eccezioni</i>	Temperatura fuori range → Il sistema richiede l'inserimento di un valore valido.

Elemento	Descrizione
<i>Nome</i>	Lampeggio LED utente
<i>ID</i>	UC6
<i>Descrizione</i>	Un LED indica visivamente che il sistema è attivo, lampeggiando a 2 Hz.
<i>Flusso eventi</i>	1. Il sistema passa in stato ATTIVO; 2. Il LED inizia a lampeggiare a 2 Hz (accensione e spegnimento ogni 250 ms); 3. Il lampeggio continua finché il sistema resta attivo.
<i>Pre-condizioni</i>	Transizione allo stato ATTIVO.
<i>Post-Condizioni</i>	Il LED lampeggia a 2 Hz.
<i>Eccezioni</i>	Il LED non lampeggia → possibile guasto o sistema in INATTIVO.

Elemento	Descrizione
<i>Nome</i>	Invio periodico parametri via UART
<i>ID</i>	UC7
<i>Descrizione</i>	Il sistema trasmette ogni 10 secondi i valori correnti dei parametri sulla UART.
<i>Flusso eventi</i>	1. Il sistema si trova in stato OPERAZIONE; 2. Ogni 10 secondi, viene inviato un pacchetto UART contenente i parametri: - stato delle luci (accese/spente); - temperatura impostata; - angolo veneziane; 3. Il ciclo continua finché il sistema resta attivo.
<i>Pre-condizioni</i>	Sistema in stato OPERAZIONE; UART configurata correttamente.
<i>Post-Condizioni</i>	Parametri aggiornati regolarmente sul monitor seriale.
<i>Eccezioni</i>	UART disconnessa → i dati non vengono ricevuti.

Elemento	Descrizione
<i>Nome</i>	Reset dei parametri
<i>ID</i>	UC8
<i>Descrizione</i>	L'utente resetta i parametri riportando il sistema allo stato INATTIVO.
<i>Flusso eventi</i>	1. Il sistema è in stato ATTIVO; 2. L'utente preme il pulsante ATTIVO; 3. Il sistema torna in stato INATTIVO; 4. Il sistema reimposta i parametri salvati;
<i>Pre-condizioni</i>	Sistema in stato ATTIVO.
<i>Post-Condizioni</i>	Sistema riportato allo stato INATTIVO.
<i>Eccezioni</i>	-

Capitolo 3

Progettazione della logica di sistema

3.1 Progettazione stateflow

La prima fase di progettazione inizia con l'implementazione della logica di controllo e di cambio di stati attraverso la libreria *Simulink* su piattaforma *Matlab*. la struttura di base del sistema è composto da DUE stati:

- INATTIVO, il sistema non effettua operazioni;
- ATTIVO;

Lo stato ATTIVO, a sua volta, si compone di tre sottostati:

- CONFIGURAZIONE, il sistema entra in stato di configurazione dei parametri;
- OPERAZIONE, il sistema attua i parametri modificati in stato CONFIGURAZIONE;
- LED_UTENTE, per la gestione del led utente;

3.1.1 Input, Output e variabili del sistema

Input:

- ATTIVA: input digitale per il pulsante ATTIVA;
- CONFIG: input digitale per il pulsante CONFIG;
- LUCI: input digitale per il pulsante LUCI;

- `LUCI_TMP`: input digitale per il pulsante `LUCI_TMP`;
- `msgIn`: input per la selezione della funzionalità del menu da modificare;
- `angolo`: input per la selezione del valore dell'angolo delle veneziane;
- `mod_luci`: input per la selezione della modalità di accensione delle luci (0: spente, 1: accese);
- `durata_luci`: input per la durata di accensione della modalità delle luci temporizzate (0: non attiva, 5 - 30: attiva);
- `temperatura`: input per la selezione della temperatura;
- `INVIO`: input digitale per il pulsante `INVIO`, che permette di inviare il valore inserito;

Variabili locali:

- `prima_attivazione`: attributo che indica se il sistema è in prima configurazione;
- `config_comp_ven`, `error_ven`: verificano il corretto inserimento dei parametri di funzionalità delle veneziane;
- `config_comp_mod_luci`, `error_mod_luci`: verificano il corretto inserimento dei parametri di funzionalità della selezione della modalità delle luci (0: spente, 1: accese);
- `config_comp_durata_luci`, `error_durata_luci`: verificano il corretto inserimento dei parametri di funzionalità della modalità temporizzata;
- `config_comp_temp`, `error_temperatura`: verificano il corretto inserimento dei parametri di funzionalità della temperatura;
- `config_comp_init`: verifica che la prima configurazione sia andata a buon fine;

Output:

- `led_utente`: output digitale per il led utente;
- `rele_luci`: output digitale per il relè;
- `temp_r`: output per il duty cycle del led rosso;
- `temp_g`: output per il duty cycle del led verde;

- `temp_b`: output per il duty cycle del led blu;
- `msgOut`: messaggio di output per il messaggio utente.
- `duty_cycle`: output per il duty cycle del servo motore;

3.1.2 Stato INATTIVO

Lo stato INATTIVO non presenta funzioni logiche particolarmente complesse, ma solamente assegnazioni ai parametri di "fondo scala" delle funzionalità del sistema.

3.1.3 Stato ATTIVO

Lo stato ATTIVO è il cuore del sistema. Esso, infatti, racchiude tutti i sottostati necessari per l'implementazione delle funzionalità. In particolare, esso è composto da due stati che lavorano in parallelo: `LED_UTENTE`, che gestisce la logica di lampeggio del led utente, e `MAIN`, che contiene le modalità di configurazione (stato CONFIGURAZIONE) e di operazione (stato OPERAZIONE).

3.1.4 Stato CONFIGURAZIONE

Lo stato CONFIGURAZIONE implementa una funzionalità vera e propria di menu per la scelta dei parametri di sistema. La logica di questo stato varia in base al valore del parametro `prima_attivazione`: se l'utente sta configurando per la prima volta il dispositivo, allora il flusso di richiesta di inserimento dei parametri delle funzionalità è sequenziale (veneziane, modalità luci, durata luci e temperatura). Se l'utente ha già completato la prima configurazione, allora verrà proposto un menu attraverso cui l'utente potrà decidere quale funzionalità settare.

Ricordiamo che, nella prima attivazione, non è consentito inserire un valore non valido. Infatti, se l'utente inserisce un valore fuori range, il sistema chiede di reinserire il valore. Nelle configurazioni successive, il meccanismo è pressoché lo stesso, con la differenza che l'utente può uscire dalla configurazione premendo il tasto `CONFIG`.

La pressione del pulsante `CONFIG`, infatti, permette di transitare allo stato `OPERAZIONE`. Nell'implementazione, però, è stato aggiunto un ulteriore vincolo: infatti, poiché l'utente alla prima attivazione deve inserire i parametri correttamente, egli non può transitare nello stato `OPERAZIONE` fintantoché non ha completato correttamente l'inserimento dei parametri (`config_comp_init = 1`).

La selezione dei parametri sarà effettuata nel range e nei vincoli imposti in precedenza.

3.1.5 Stato OPERAZIONE

Lo stato OPERAZIONE prevede l'esecuzione simultanea di quattro sotto-modalità: riscaldamento, illuminazione, controllo delle veneziane e notifica dello stato. L'obiettivo principale è gestire tutte le funzionalità in parallelo, così da sincronizzare la regolazione di tutti gli attributi di sistema.

3.2 Implementazione simulink

Lo stato INATTIVO, in figura 3.2, prevede solo un messaggio di uscita su `msgOut` con la stringa "Premere il pulsante ATTIVA per attivare il dispositivo" e vengono assegnati tutti gli attributi in configurazione di *low bound* per tutti i parametri logici e di controllo.

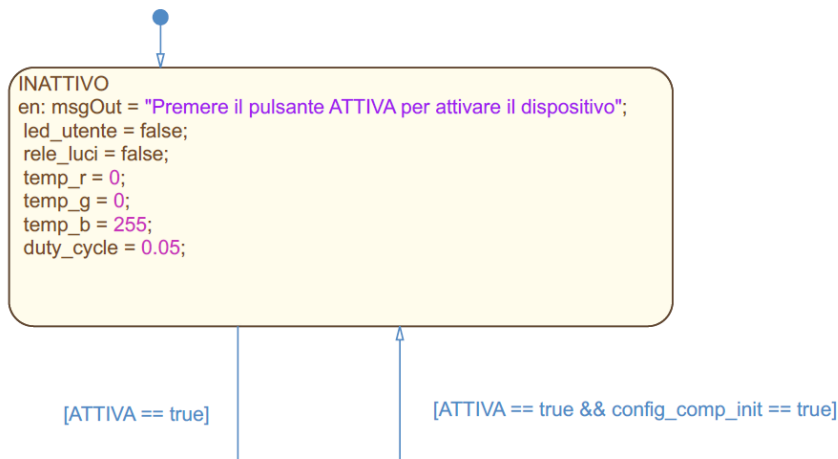


Figura 3.1: Costruzione dello stato INATTIVO.

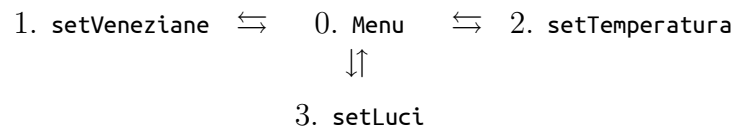
Lo stato CONFIG prevede un menù per la selezione dei parametri, creato *ad hoc*, in modo da gestire le configurazioni a runtime.

La prima configurazione inizia con la visualizzazione del messaggio in uscita (su `msgOut`) con la stringa "Benvenuto nell'ambiente domotico! Alla prima attivazione, è necessario configurare tutte le funzionalità". Come precedentemente illustrato nel paragrafo 3.1.4, alla prima attivazione l'inserimento dei parametri è sequenziale:

angolo → mod_luci → durata_luci → temperatura

Il passaggio dalla selezione di un parametro all'altro avviene impostando il valore desiderato e premendo il pulsante **INVIO**. Nella prima configurazione, alla fine del settaggio di tutti i parametri, il sistema stampa in uscita il messaggio *"Configurazione iniziale completata! Per entrare nella modalità operativa, premere il tasto CONFIG"*. Inoltre, viene impostato il valore di `config_comp_init` pari a 1, in modo da permettere il passaggio allo stato OPERATIVO, e impostato il valore del flag `prima_attivazione` pari a 0, in modo da abilitare il passaggio allo stato INATTIVO.

Dalla seconda configurazione in poi, si considera anche l'ingresso `msgIn` dove si immette un carattere da '1' a '3' per scegliere l'attributo da modificare e non appena esso viene impostato si ritorna nella home del menu con la possibilità di assegnare altri attributi o tornare nella fase di esecuzione delle funzionalità.



Da notare è l'utilizzo di flag quali:

- `config_comp_ven`, `error_ven`: verificano il corretto inserimento dei parametri di funzionalità delle veneziane;
- `config_comp_mod_luci`, `error_mod_luci`: verificano il corretto inserimento dei parametri di funzionalità della selezione della modalità delle luci (0: spente, 1: accese);
- `config_comp_durata_luci`, `error_durata_luci`: verificano il corretto inserimento dei parametri di funzionalità della modalità temporizzata;
- `config_comp_temp`, `error_temperatura`: verificano il corretto inserimento dei parametri di funzionalità della temperatura;

In particolare, i flag `config_comp_...` vengono settati a 1 nel caso in cui il valore inserito sia contenuto nel range valido. I flag `error_...` sono necessari per l'implementazione della richiesta ciclica di reinserimento del valore in caso di valori errati.

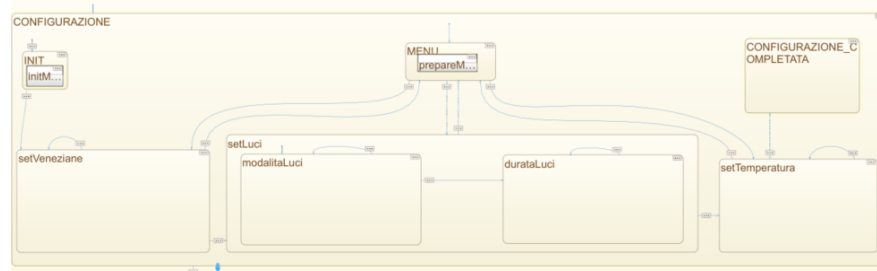


Figura 3.2: Costruzione dello stato CONFIGURAZIONE. Per maggiori dettagli, aprire il file **50_Group31_ProjectWork_EmbeddedSystem_Chart.pdf**

Lo stato di OPERAZIONE mette in atto tutte le modifiche avvenute nella fase di configurazione. In particolare:

- Lo stato LUCI, implementato come macchina a stati finiti che fonde la logica delle FSM in figura 2.1 e 2.2;
- Lo stato VENEZIANE, che grazie alla funzione `opVeneziane()`, imposta il preset corretto al servo motore;
- Lo stato TEMPERATURA, che grazie alla funzione `opTemperatura()`, imposta la modulazione corretta ai led RGB;
- Lo stato NOTIFICA_STATO, aggiorna il messaggio UART in uscita su `msgOut` ogni 10 sec;

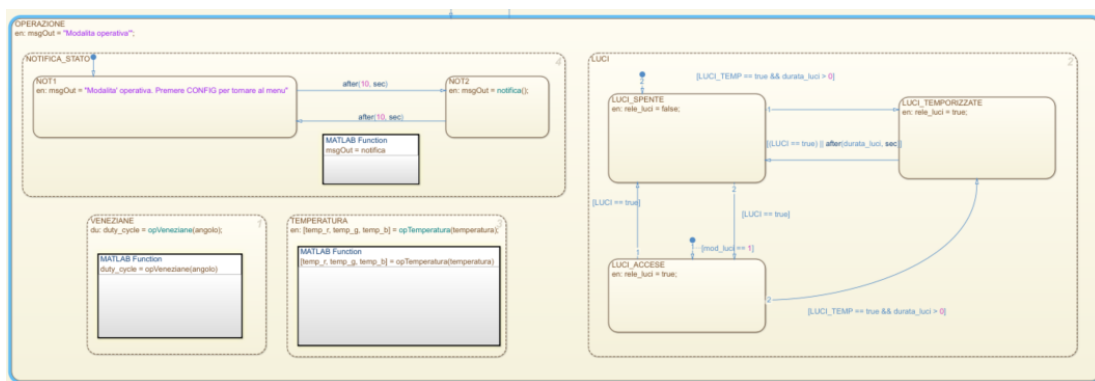


Figura 3.3: Costruzione dello stato OPERATIVO, che gestisce la logica parallela delle tre funzionalità.

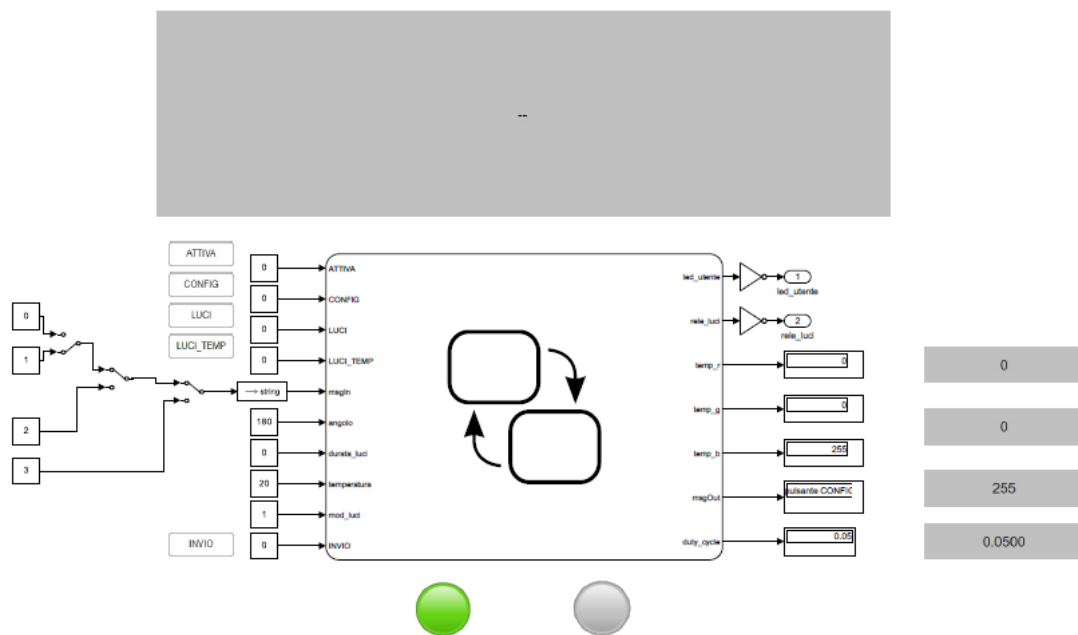


Figura 3.4: Ingressi e uscite per la simulazione del chart.

Capitolo 4

Implementazione del sistema domotico

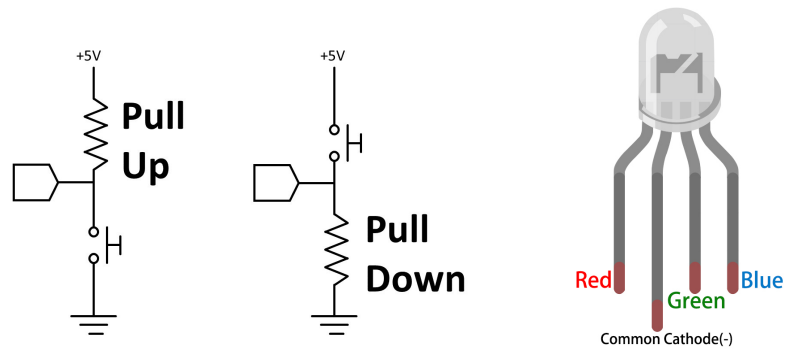
4.1 Schema di collegamento

In questa sezione si presenta lo schema di collegamento tra il nucleo SMT32 e i componenti di sensoristica presentati nell'introduzione del progetto. In particolare, in figura 4.1 sono elencati i PIN utilizzati per il collegamento tra la scheda e i vari componenti utilizzati.

Dispositivo	PIN	Descrizione
LED RGB colore rosso	PA6	TIM 3 PWM CH1
LED RGB colore verde	PC7	TIM 3 PWM CH2
LED RGB colore blue	PC9	TIM 3 PWM CH4
LED utente	PA5	–
Servo motore SG90	PB6	TIM 4 PWM CH1
Relay	PC2	–
Pulsante CONFIG	PB0	–
Pulsante ATTIVA	PC13	User button
Pulsante LUCI	PC10	–
Pulsante LUCI_TMP	PA15	–
RX	PA3	LPUART RX
TX	PA2	LPUART TX

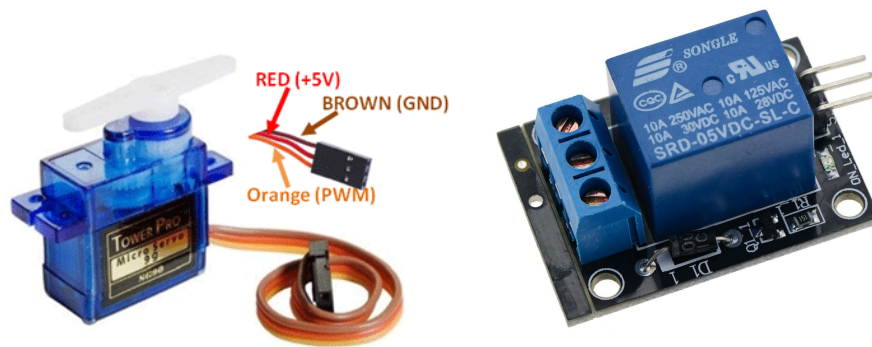
Tabella 4.1: Mappatura dei pin rispetto alla sensoristica.

I pulsanti posizionati sulla breadboard sono stati collegati con resistenza di pull-down di $10k\Omega$ anche se è possibile configurare direttamente da cubeMX. Per i LED, invece, si è scelto come valore del resistore quello di 220Ω .



(a) Differenza di configurazione in pull-up o pull-down per il pulsante.

(b) Pinout del led RGB.



(c) Pinout del servo motore SG90.

(d) Modulo relè 5V.

Figura 4.1: Componenti utilizzati nel sistema

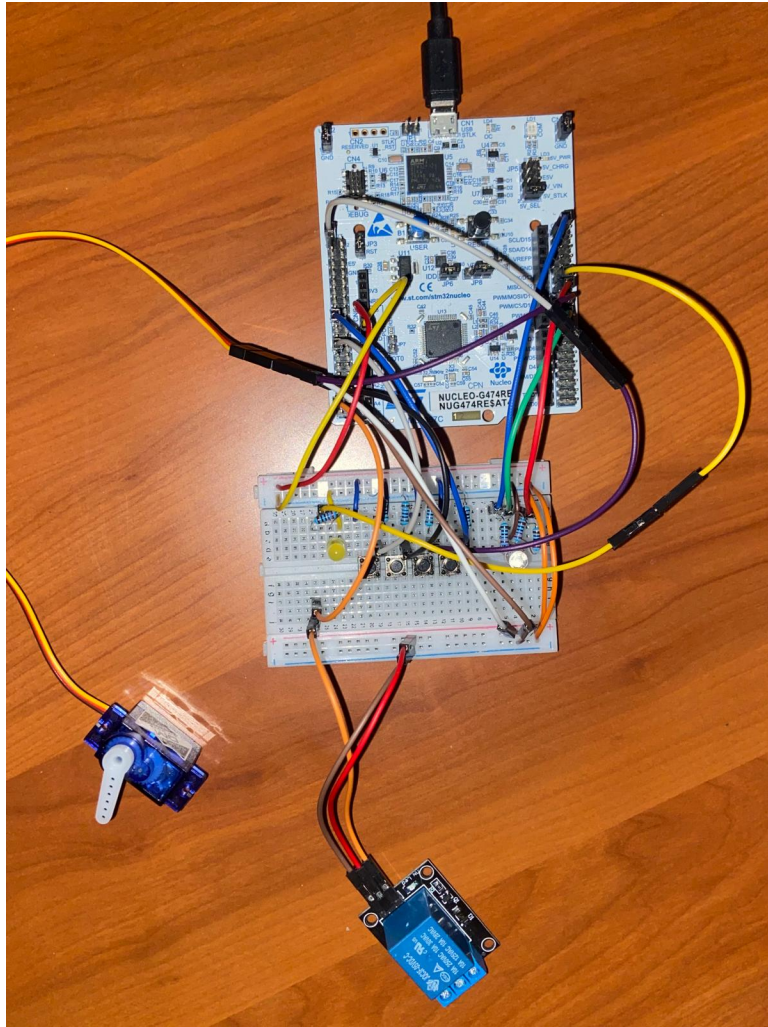


Figura 4.2: Costruzione del circuito domotico nel mondo reale.

4.2 Implementazione Simulink con STM32

La costruzione del chart Stateflow per la configurazione con microcontrollore STM32 prevede delle differenze effettuate a causa dell'interfacciamento con UART e per le problematiche di velocità eccessiva di clock di sistema. Nella figura 4.3, la funzionalità dello stato INATTIVO è rimasta inalterata, Tuttavia prima di esso è stato inserito uno stato CLEAR per il reset del terminale UART ed è stata inserita una funzione per la conversione da stringa a double per l'impostazione dei parametri di sistema da messaggio UART.

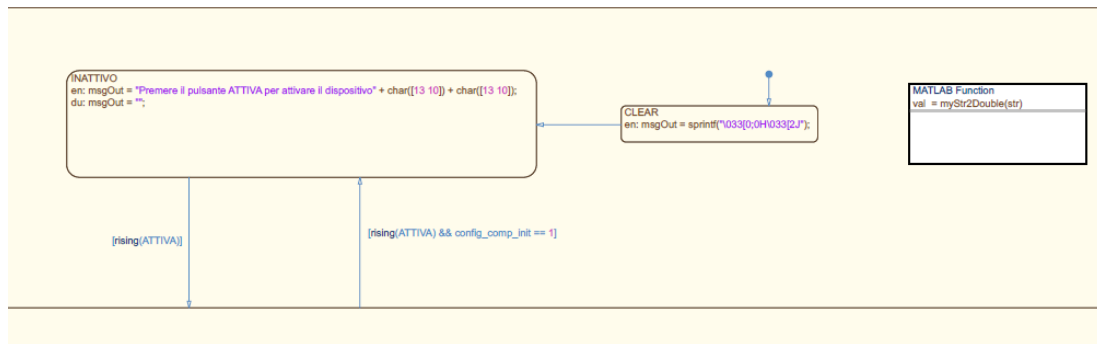


Figura 4.3: Sezione della fase INATTIVO, del blocco per la pulizia del terminale e della funzione di conversione myStr2Double.

Una modifica sostanziale è stata introdotta nella sezione di configurazione, a causa delle difficoltà riscontrate nella gestione dei caratteri da terminale UART, che impedivano un'esecuzione affidabile delle fasi di impostazione dei parametri. Per risolvere il problema, sono stati adottati alcuni accorgimenti, tra cui l'introduzione di un segnale di controllo denominato `len`, utilizzato per leggere la lunghezza della stringa UART ricevuta.

Un'altra differenza sostanziale rispetto alla simulazione iniziale riguarda la gestione del comando di conferma. Mentre nell'implementazione puramente Simulink era stato utilizzato un pulsante simulato per rappresentare il tasto INVIO, nell'implementazione fisica con STM32 si è optato per l'utilizzo diretto del carattere ASCII corrispondente al tasto INVIO della tastiera (`char(13)`).

Questo nuovo input, integrato nel blocco di lettura UART, consente di attivare le logiche di decodifica solo sul fronte di salita del segnale `len`, garantendo un'elaborazione sicura e sincronizzata dei dati in ingresso. Tali accorgimenti hanno migliorato significativamente la robustezza del sistema, in particolare durante la fase di prima configurazione, che risulta ora più stabile ed efficiente anche rispetto alla precedente implementazione esclusivamente in Simulink.

La logica di scarto dei valori non corretti è rimasta inalterata, nonostante sia cambiata invece l'implementazione, che viene mostrata in figura 4.4. In particolare, quando viene inserito un valore errato viene stampato a video il messaggio *"Valore MODALITA ... non valido. Premi INVIO per reinserirlo"*.

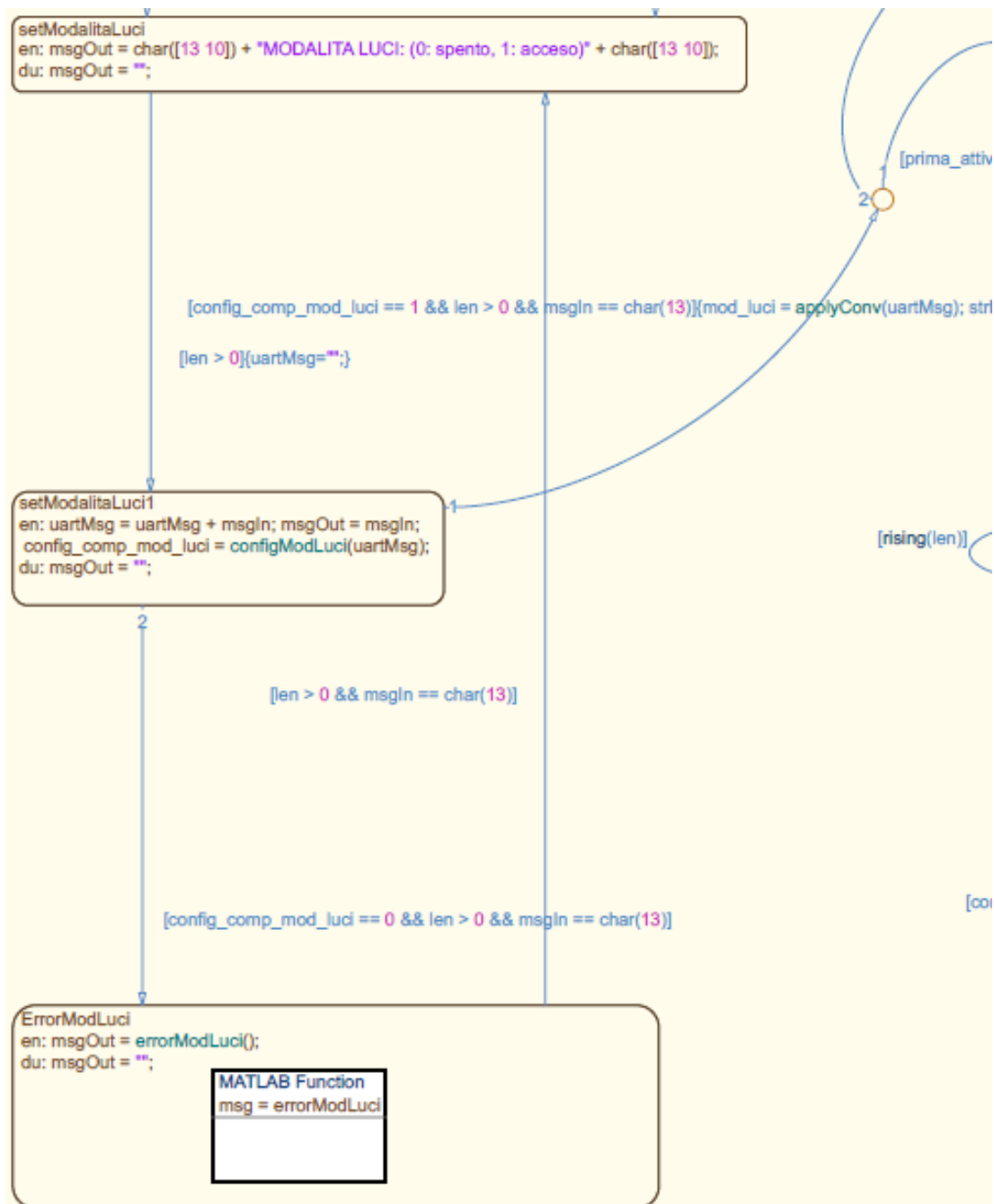


Figura 4.4: Esempio della nuova implementazione per la logica di scarto

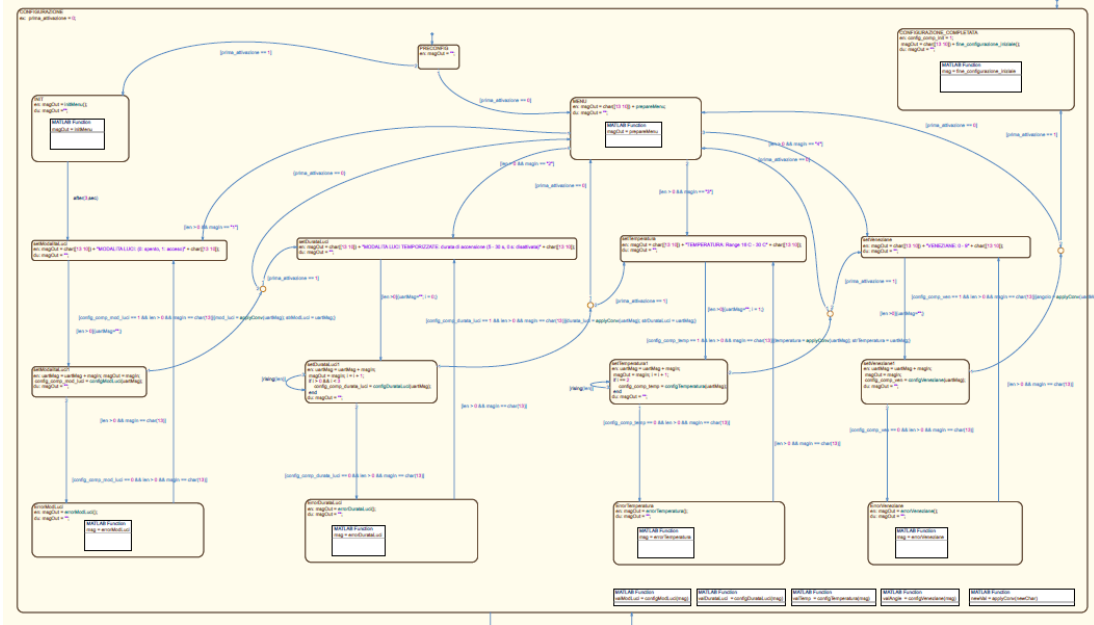


Figura 4.5: Visualizzazione della fase di configurazione (ricostruita) più la logica per la prima configurazione.

In riferimento alla figura 4.6, il chart è uguale a quello utilizzato per la sola implementazione in Simulink, tuttavia al posto dei blocchi precedenti sono stati inseriti dei costrutti dedicati alle uscite e alle entrate analogico/digitali della STM32G474RET3. Essi sono:

- *Digital Port Read*: utilizzato per la lettura di segnali digitali in ingresso da terminali GPIO del microcontrollore, come nel caso dei pulsanti;
- *Digital Port Write*: consente la scrittura di segnali digitali in uscita, ad esempio per controllare il relè o il LED utente;
- *PWM Output*: impiegato per generare segnali PWM su specifici pin, utile per il controllo dell'intensità luminosa del LED RGB e del servo motore;
- *UART/USART Read*: utilizzato per ricevere dati dalla comunicazione seriale asincrona, come comandi o configurazioni inviati dall'utente tramite terminale;
- *UART/USART Write*: consente la trasmissione di dati via seriale, ad esempio per l'invio periodico dello stato del sistema o messaggi di feedback.

In più, sono stati adoperati per le funzionalità UART i blocchi *ASCII* $\rightarrow$ *String* e *String* $\rightarrow$ *ASCII*.

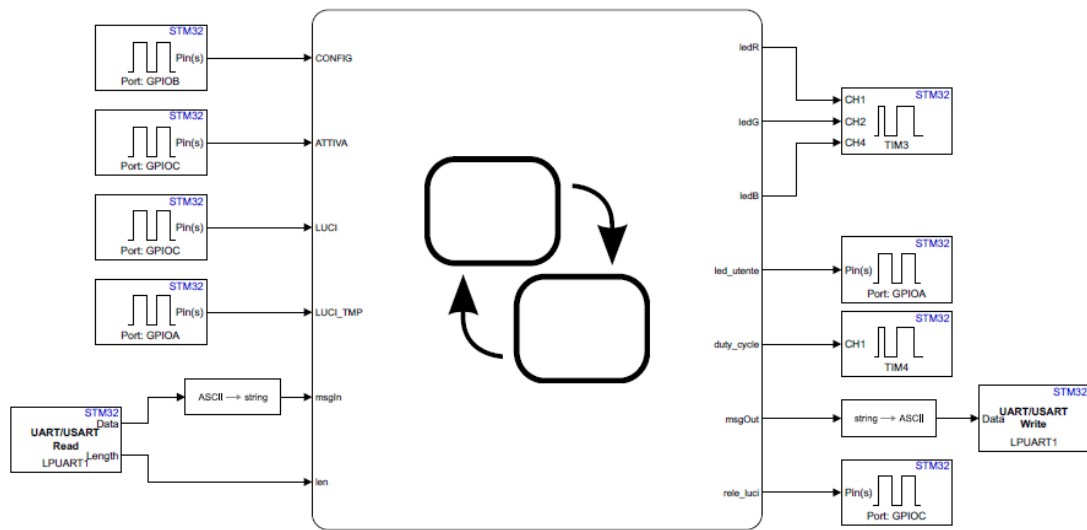


Figura 4.6: Chart Simulink compreso di blocchi per l'input e output riguardanti la scheda STM32G474RE3.

4.2.1 Impostazioni ST CubeMX

Per il flash del programma compilato su Simulink è stata necessaria la creazione di un file `.ioc` per il setting dei moduli di GPIO, LPUART (Low Power UART) e Timing. Si specifica che il clock generale del sistema è stato impostato a $16MHz$.

Setting GPIO

Gli *input/output* di tipo **GPIO** (General Purpose Input/Output) sono stati impiegati principalmente per la gestione della pulsantiera e di alcuni segnali digitali in uscita. In particolare:

- I pulsanti **ATTIVA**, **CONFIG**, **LUCI** e **LUCI_TMP** sono stati configurati come **input digitali** con resistenza di *pull-up* attiva. Il pulsante **ATTIVA** corrisponde al tasto integrato nel microcontrollore (pin **PC13**), mentre gli altri sono pulsanti esterni connessi rispettivamente ai pin **PB0**, **PC10** e **PC15**;
- Il **relè**, utilizzato per il controllo del circuito luci, è collegato a un **pin di uscita digitale** (GPIO output) sul pin **PC2**;
- Il **LED utente**, impiegato per la segnalazione visiva dello stato attivo del sistema (lampeggio a 2 Hz, ottenuto tramite blocco simulink e non temporizzazione diretta), è il LED integrato sulla scheda e si trova sul pin **PA5**.

Setting LPUART

Il modulo **LPUART1**, configurato in modalità *asincrona*, è stato impiegato per la comunicazione seriale tra il microcontrollore e un terminale esterno, in particolare tramite il software *PuTTY*. La periferica è stata utilizzata sia per la trasmissione che per la ricezione dei dati, permettendo la configurazione e il monitoraggio dello stato del sistema in tempo reale. I principali parametri di configurazione sono i seguenti:

- **Baud Rate:** 115200 bit/s;
- **Word Length:** 8 bit (senza bit di parità);
- **Parity:** None;
- **Stop Bits:** 1.

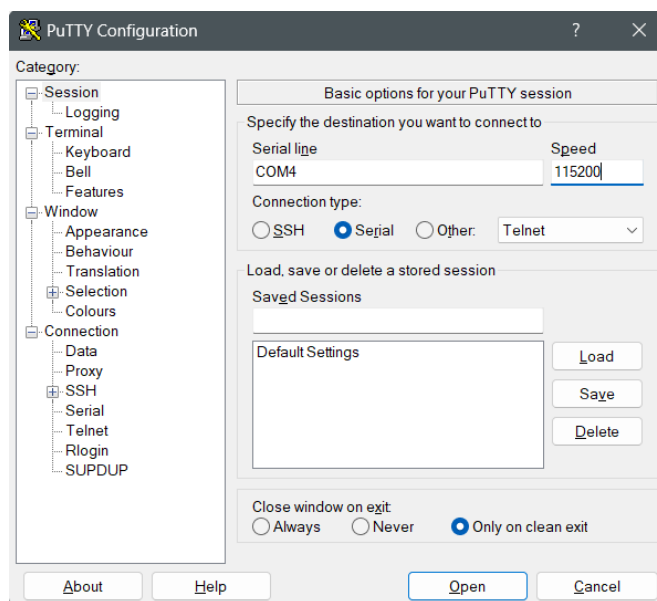


Figura 4.7: Configurazione terminale UART PuTTY.

Setting Timer

Il **Timer3** è impiegato per il controllo della luminosità del LED RGB attraverso la tecnica di *PWM dimming*. A tal fine, vengono utilizzati i canali **CH1**, **CH2** e **CH4**, associati rispettivamente alle componenti rossa, verde e blu del LED. L'obiettivo è generare un segnale PWM sufficientemente veloce da non causare sfarfallii visibili all'occhio umano. L'intervallo di aggiornamento del timer, ovvero la frequenza dell'evento di aggiornamento del contatore (Update Event), è calcolato secondo la seguente formula:

$$\text{UpdateEvent} = \frac{\text{Timer}_{\text{clock}}}{(\text{Prescaler} + 1)(\text{Period} + 1)} = \frac{16 \cdot 10^6}{(15 + 1)(99 + 1)} = 100 \text{ Hz}$$

Con un *Prescaler* pari a 15 e un *Period* pari a 99, si ottiene una frequenza di aggiornamento di 100Hz, che consente un controllo fine dell'intensità luminosa mediante variazione del *duty cycle* su ciascun canale.

Il **Timer4** è invece dedicato al controllo del *servo motore*, utilizzato per regolare l'angolo di apertura delle veneziane. Viene impiegato il canale **CH1**, che genera un segnale PWM con una frequenza e un duty cycle adatti a comandare il posizionamento dell'albero del motore. L'Update Event viene calcolato con la stessa formula, ma con valori differenti per prescaler e periodo:

$$\text{UpdateEvent} = \frac{\text{Timer}_{\text{clock}}}{(\text{Prescaler} + 1)(\text{Period} + 1)} = \frac{16 \cdot 10^6}{(319 + 1)(999 + 1)} = 50 \text{ Hz}$$

In questo caso, si ottiene una frequenza di aggiornamento di 50 Hz, compatibile con la maggior parte dei servo motori standard, che richiedono impulsi di controllo ogni 20 ms. Il duty cycle viene modulato tra circa 5% e 10% per coprire l'intervallo di rotazione da 0° a 180°.

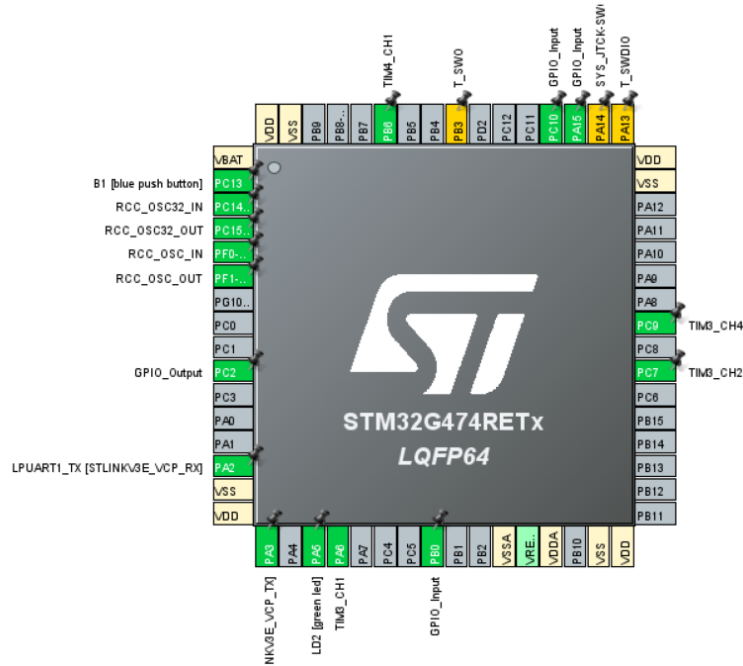
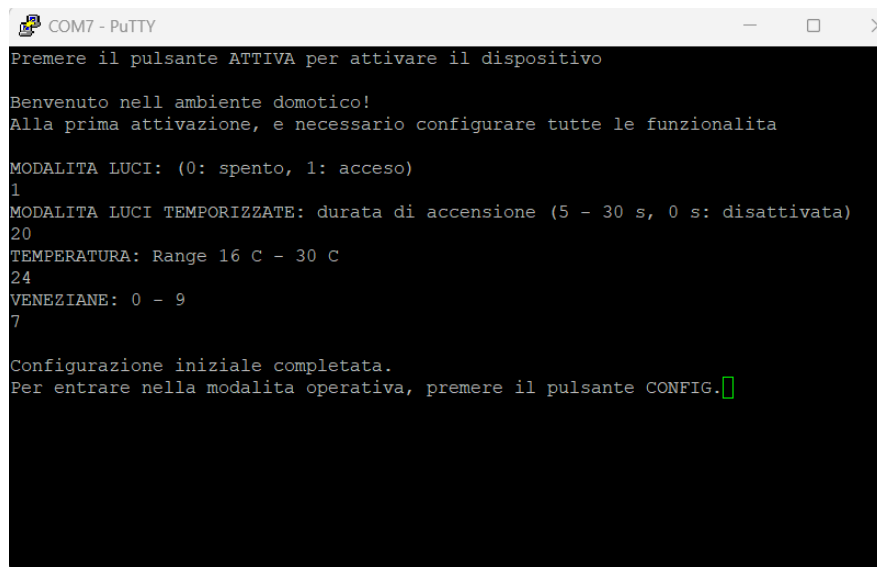


Figura 4.8: Pinout della scheda STM32G474RET3 con relativi setting per ogni pin.

4.3 Fase di testing

Dopo aver validato singolarmente ciascuna funzionalità del sistema tramite ambiente Simulink, attraverso veri e propri *unit test*, si è proceduto con l'integrazione progressiva dei componenti in sottosistemi via via più complessi. Questo ha permesso l'esecuzione di test di tipo *system testing*, al fine di verificare il comportamento del sistema nel suo complesso.

In figura 4.9 è riportato un esempio di configurazione iniziale avvenuta con successo tramite terminale seriale.



```
COM7 - PuTTY
Premere il pulsante ATTIVA per attivare il dispositivo

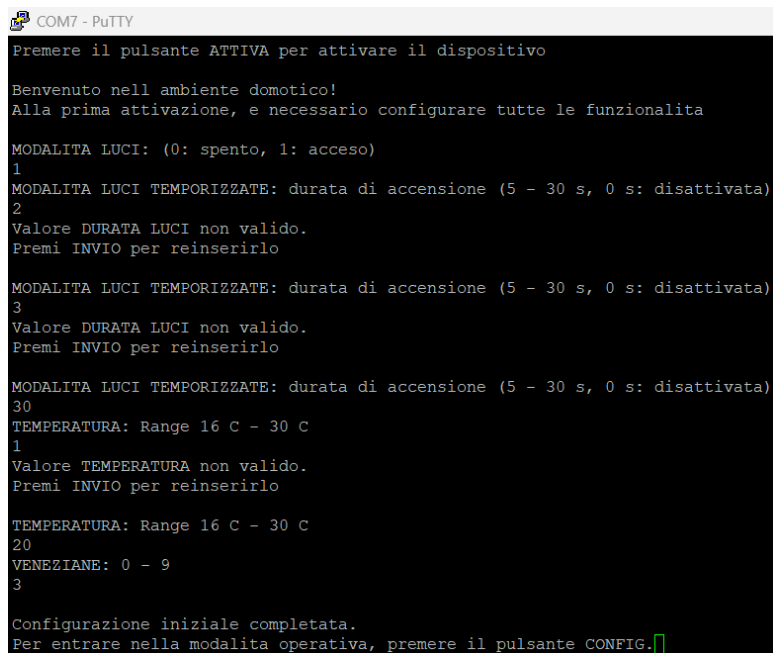
Benvenuto nell'ambiente domotico!
Alla prima attivazione, è necessario configurare tutte le funzionalita

MODALITA LUCI: (0: spento, 1: acceso)
1
MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
20
TEMPERATURA: Range 16 C - 30 C
24
VENEZIANE: 0 - 9
7

Configurazione iniziale completata.
Per entrare nella modalita operativa, premere il pulsante CONFIG.█
```

Figura 4.9: Esempio di prima configurazione inserita correttamente tramite terminale.

Una delle funzionalità più critiche sottoposte a test è proprio la **gestione di una configurazione errata in fase iniziale**. Come già accennato, il sistema, in caso di valore errato, richiede all'utente di inserire un valore valido, e questo stato persiste fintantoché esso non viene inserito, come mostrato in figura 4.10.



```

COM7 - PuTTY
Premere il pulsante ATTIVA per attivare il dispositivo

Benvenuto nell'ambiente domotico!
Alla prima attivazione, e necessario configurare tutte le funzionalita

MODALITA LUCI: (0: spento, 1: acceso)
1
MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
2
Valore DURATA LUCI non valido.
Premi INVIO per reinserirlo

MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
3
Valore DURATA LUCI non valido.
Premi INVIO per reinserirlo

MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
30
TEMPERATURA: Range 16 C - 30 C
1
Valore TEMPERATURA non valido.
Premi INVIO per reinserirlo

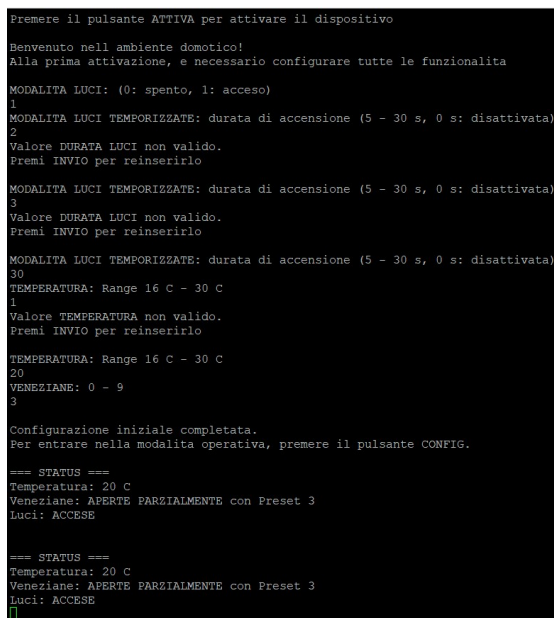
TEMPERATURA: Range 16 C - 30 C
20
VENEZIANE: 0 - 9
3

Configurazione iniziale completata.
Per entrare nella modalita operativa, premere il pulsante CONFIG.

```

Figura 4.10: Gestione dell'inserimento di configurazioni invalide in fase iniziale.

Infine, per verificare la corretta trasmissione periodica del messaggio UART di aggiornamento, si è proceduto con l'attivazione di una funzionalità (in questo caso l'accensione manuale delle luci) e si è attesa la stampa automatica del log ogni 10 secondi. Il risultato è illustrato in figura 4.11.



```

Premere il pulsante ATTIVA per attivare il dispositivo

Benvenuto nell'ambiente domotico!
Alla prima attivazione, e necessario configurare tutte le funzionalita

MODALITA LUCI: (0: spento, 1: acceso)
1
MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
2
Valore DURATA LUCI non valido.
Premi INVIO per reinserirlo

MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
3
Valore DURATA LUCI non valido.
Premi INVIO per reinserirlo

MODALITA LUCI TEMPORIZZATE: durata di accensione (5 - 30 s, 0 s: disattivata)
30
TEMPERATURA: Range 16 C - 30 C
1
Valore TEMPERATURA non valido.
Premi INVIO per reinserirlo

TEMPERATURA: Range 16 C - 30 C
20
VENEZIANE: 0 - 9
3

Configurazione iniziale completata.
Per entrare nella modalita operativa, premere il pulsante CONFIG.

=== STATUS ===
Temperatura: 20 C
Veneziane: APERTE PARZIALMENTE con Preset 3
Luci: ACCESE

=== STATUS ===
Temperatura: 20 C
Veneziane: APERTE PARZIALMENTE con Preset 3
Luci: ACCESE

```

Figura 4.11: Messaggio UART periodico di aggiornamento dei parametri di sistema.

Capitolo 5

Conclusioni

Il presente progetto ha dimostrato l'efficacia dell'approccio metodologico V-Model nell'ambito dello sviluppo di sistemi embedded, evidenziando come l'integrazione di metodologie model-based design con piattaforme hardware moderne possa portare alla realizzazione di soluzioni domotiche robuste e affidabili.

5.1 Risultati Raggiunti

Il sistema domotico sviluppato ha soddisfatto pienamente tutti i requisiti funzionali e non funzionali definiti nella fase di analisi. In particolare:

5.2 Funzionalità Implementate

- **Controllo delle luci:** Gestione completa dell'illuminazione con modalità manuale e temporizzata, inclusa la gestione dei timer e l'interruzione automatica;
- **Sistema di riscaldamento:** Simulazione termica con feedback visivo tramite LED RGB e codifica cromatica a cinque livelli;
- **Gestione veneziane:** Controllo automatizzato del servo motore con posizionamento discreto in step di 20°;
- **Interfaccia UART:** Configurazione completa dei parametri via terminale seriale con validazione degli input;
- **Sistema di stati:** Implementazione robusta della macchina a stati con gestione della prima configurazione e configurazioni successive.

5.3 Aspetti Tecnici

- Transizione efficace dalla simulazione Simulink all'implementazione hardware STM32;
- Gestione ottimizzata delle comunicazioni UART con controllo del flusso dati;
- Implementazione PWM per il controllo preciso di LED RGB e servo motore;
- Architettura modulare che consente l'esecuzione parallela delle funzionalità.

5.4 Innovazioni Introdotte

Durante la transizione dall'ambiente simulativo all'implementazione fisica sono state apportate diverse migliorie:

1. **Gestione Input UART:** Sostituzione del pulsante INVIO simulato con il carattere ASCII reale (`char(13)`), migliorando significativamente l'usabilità del sistema;
2. **Controllo Flusso Dati:** Introduzione del segnale di controllo `len` per la sincronizzazione della lettura UART;
3. **Robustezza Sistema:** Implementazione di logiche di validazione più stringenti per la gestione degli errori di configurazione;
4. **Ottimizzazione Clock:** Configurazione del sistema a 16MHz per garantire prestazioni ottimali.

5.5 Validazione e Testing

La fase di testing ha confermato la solidità del sistema attraverso:

- **Unit Testing:** Validazione di ogni singola funzionalità;
- **Integration Testing:** Verifica dell'interoperabilità tra componenti hardware e software;
- **System Testing:** Validazione del comportamento del sistema completo;
- **Stress Testing:** Test di robustezza con configurazioni errate e transizioni ripetute.

I risultati hanno dimostrato la capacità del sistema di gestire correttamente scenari d'uso reali, inclusi casi limite e situazioni di errore.

5.6 Vantaggi dell'Approccio Model-Based

L'utilizzo di Simulink e Stateflow ha permesso di:

- **Anticipare problematiche critiche** attraverso simulazioni virtuali prima della prototipazione fisica;
- **Ridurre significativamente i costi** di correzione degli errori identificando anomalie nelle fasi iniziali;
- **Garantire tracciabilità** tra requisiti di alto livello e implementazione dettagliata;
- **Accelerare il processo di sviluppo** grazie alla generazione automatica del codice.

5.7 Considerazioni Finali

Il progetto ha dimostrato come l'approccio metodico del V-Model, combinato con strumenti di model-based design, consenta di mantenere un elevato standard qualitativo durante tutto il ciclo di sviluppo, riducendo significativamente il rischio di difetti critici nelle fasi finali del progetto.

L'esperienza acquisita evidenzia l'importanza di una progettazione sistematica nell'ambito dei sistemi embedded, dove la complessità derivante dall'intreccio tra componenti hardware e software richiede metodologie strutturate per garantire risultati affidabili e manutenibili.

Il sistema realizzato non solo soddisfa i requisiti tecnici specificati, ma pone anche le basi per un'architettura scalabile e modulare, dimostrando il potenziale delle tecnologie embedded moderne nell'automazione domestica intelligente.